

Towards Generalisation of AI4Science Systems: Growing a Verified, Reusable Tool Ecosystem from Scientific Repositories

Anonymous submission

Abstract

AI4Science systems run their experiments in one of two limiting ways: a costly coding agent that re-writes a solution on every call—so repeated runs can differ in output *format* and *value*—or a small, fixed set of tools that cannot cover the long tail of scientific software. We instead grow a *verified, reusable catalogue* of tools built from scientists’ own code. Our system’s experiment module converts any scientific repository into a served, verified Model Context Protocol (MCP) tool and reuses it across studies. The converter is a five-agent pipeline with code-enforced gates that build isolated environments and validate each tool by live invocation against held-out gold. On a gold-validated benchmark we show *when* the catalogue helps—reproducible repeated calls, wider coverage, and cost amortised over many calls—and, in the interest of honesty, where it does not: modern coding agents already match its per-task accuracy. We release all code, tools, and runs.

1 Introduction

Autonomous “AI4Science” systems now do more than propose hypotheses: they plan and *execute* numerical experiments, from chemical synthesis to computational biology (Boiko et al. 2023; M. Bran et al. 2024; Gottweis et al. 2025; Swanson et al. 2025). Yet almost all of them run the experiment step in one of two regimes. Either a general-purpose *coding agent* re-derives a solution from scratch on every call—installing dependencies, writing code, and executing it in a sandbox (Lu et al. 2024; Schmidgall et al. 2025)—or the system is wired to a *small, fixed set* of ready-made tools or servers hand-built by its authors (Boiko et al. 2023; M. Bran et al. 2024). Both regimes place a hard ceiling on the experiment layer.

The coding-agent regime is expensive and, more fundamentally, *stochastic*: the same experiment re-derived twice can differ not only in cost but in the *format* and *value* of its output. This is not merely sampling noise that a lower temperature removes—recent work shows that even temperature-0 LLM inference is non-deterministic for architectural reasons (He and Thinking Machines Lab 2025),

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

and the broader reproducibility record of computational science is poor: fewer than one in ten biomedical Jupyter notebooks reproduce their published results (Samuel and Mitichen 2024). Coding agents also fail disproportionately on exactly the obscure, under-documented research repositories that most need automation (Bogin et al. 2024; Siegel et al. 2024; Starace et al. 2025). The fixed-tool regime avoids stochasticity but cannot scale: curated scientific tool coverage is thin, and much of the growing Model Context Protocol (MCP) server ecosystem is low-quality or unmaintained (Hou et al. 2025; Anonymous 2025). In both regimes, the set of experiments a system can *reliably* run is narrow.

Our starting observation is that an extensible ecosystem of *verified* scientific tooling would relax this ceiling. If experiments could be executed through a growing catalogue of tools built from domain scientists’ own code—validated once and then reused—the experiment layer would lean on expert code rather than on stochastic regeneration, cover the long tail of repositories, and, crucially, yield a *reproducible* record for downstream experiments to build on. This is the logic of tool-making and skill-library agents (Cai et al. 2023; Yuan et al. 2024; Wang et al. 2023), but applied to whole scientific repositories and integrated into a system that plans across many experiments.

We realise this idea as the *experiment module* of a general AI4Science system with a planner, an action graph, and an executor. Faced with an experiment, the executor first looks for a suitable tool in a shared catalogue; if none exists, it invokes our converter, ALEMBIC, which turns an arbitrary GitHub repository into a served, gate-verified MCP tool, and then calls that tool to run the experiment. Conversion is a five-agent pipeline interleaved with *code-enforced gates* (the LLM proposes, deterministic code disposes): it builds an isolated environment, writes each tool as a plain function, and validates it by *live invocation on held-out inputs*—not by an import-smoke check or an LLM judge. Every accepted tool enters the catalogue, so the system’s experimental reach grows monotonically as more repositories are converted.

We are deliberately careful about what this does and does not buy, because the mechanism of “convert a repository into a verified served tool” is now occupied by strong concurrent work—ToolMaker (Wölflein et al. 2025), Paper2Agent (Miao and Zou 2025), and the ToolUniverse catalogue (Gao et al. 2025). Our contribution is not that conver-

sion is possible, but a *generalising system that consumes a growing verified catalogue across domains and studies*, together with an honest empirical account of when this helps. Concretely, we contribute:

- A **system** that integrates repository-to-tool conversion into a planner–action-graph–executor loop with a shared, growing tool catalogue, turning one-off experiment code into reusable, composable infrastructure.
- **ALEMBIC**, a repository-to-verified-MCP converter whose code-enforced gates and always-split environments make catalogue entries reproducible by construction, and whose validation resists the hollow/mock “passing” tests that inflate other pipelines.
- An **empirical characterisation** on a gold-validated scientific benchmark of *when* a verified reusable catalogue improves reproducibility, coverage, and amortised cost—and, in the interest of scientific integrity, where it does *not*: on the repositories we could test head-to-head, a modern coding agent matches the converted tools’ per-task capability and sets up even gated, version-pinned environments reliably. The catalogue’s value is therefore reproducibility, coverage growth, and amortised cost across a many-call system—not per-task capability—and we frame the system as *hybrid*: catalogue-first, with a coding-agent fallback for genuinely novel experiments.

2 Related Work

Autonomous scientists. Systems such as Coscientist (Boiko et al. 2023), ChemCrow (M. Bran et al. 2024), the AI Scientist (Lu et al. 2024), Google’s AI co-scientist (Gottweis et al. 2025) and the Virtual Lab (Swanson et al. 2025) plan and run scientific work end to end. Their experiment layers are exactly the two regimes we target: a small fixed toolset (18 hand-built tools in ChemCrow; a bespoke pipeline in the Virtual Lab) or an open-ended coding agent that regenerates code per run. We keep the planning apparatus and replace the experiment layer with a growing, verified catalogue.

Coding agents on research repositories. Benchmarks that ask agents to set up and run real repositories report low success—SUPER (Bogin et al. 2024), CORE-Bench (Siegel et al. 2024) and PaperBench (Starace et al. 2025)—motivating a mechanism that pays the setup cost once. Separately, LLM execution is non-deterministic even at temperature 0 (He and Thinking Machines Lab 2025), and computational-science artefacts reproduce poorly (Samuel and Mietchen 2024); agentic pass^k degrades over repeated runs (Yao et al. 2024). We lead with this *reproducibility* argument rather than cost, since inference cost is falling rapidly and is a weaker long-run differentiator.

Tool-making and skill libraries. Making a reusable tool once and calling it thereafter reduces cost and variance (Cai et al. 2023); CRAFT curates a validated, retrievable toolset (Yuan et al. 2024); Voyager grows a library of verified executable skills (Wang et al. 2023); Toolformer learns API use (Schick et al. 2023). We apply this principle at the granularity of whole scientific repositories and under code-enforced verification.

Repository/paper-to-MCP conversion (closest work). The mechanism of converting code into a served tool is now well populated. **ToolMaker** (Wölflein et al. 2025) turns a repository and task into a tool via closed-loop unit-test self-correction and ships the TM-Bench gold suite we evaluate on. **Paper2Agent** (Miao and Zou 2025) builds an MCP server from a paper plus code and locks tools to reproduce reference results. **ToolUniverse** (Gao et al. 2025) assembles a 600+ tool scientific ecosystem and synthesises tools from natural-language descriptions. Code2MCP (Ouyang et al. 2025) and ToolRosella (Di et al. 2026) convert repositories at scale but validate by import-smoke or an LLM judge rather than held-out gold. Our delta is threefold: (i) we *consume* a growing catalogue inside a general planner–executor system across domains and studies, rather than producing tools as an end in itself; (ii) our validation is live invocation against held-out gold with a guard against hollow/mock tests, stronger than import-smoke or LLM-judge; and (iii) our always-split environments and subprocess invocation contract make catalogue entries reproducible by construction. We position *hybrid* (catalogue-first, coding-agent fallback), not tools-versus-code—consistent with evidence that code actions can beat rigid tool-calling.

Scientific benchmarks. Many benchmarks pair scientific code with tasks, but few provide strict, reproducible gold on *tool* outputs. TM-Bench (Wölflein et al. 2025) validates converted tools by `allclose` against reference arrays over multiple input cases—the closest fit to our setting. ML-Bench (Tang et al. 2024a) and CORE-Bench (Siegel et al. 2024) add scale and reproducibility of diverse repositories; ScienceAgentBench (Chen et al. 2025), SciCode (Tian et al. 2024), BioCoder (Tang et al. 2024b) and Research-CodeBench evaluate scientific code generation but at function or program granularity. ToolRosella (Di et al. 2026) offers the broadest repository coverage (122 repositories) but ships no gold answers—its downstream scores come from execution success plus an LLM judge—so we use it for coverage breadth, not correctness. We adopt TM-Bench as our primary gold benchmark and draw on the others for breadth.

3 System

4 Alembic

5 Experimental Setup

Benchmark. We evaluate on **TM-Bench** (Wölflein et al. 2025), the only public benchmark whose design mirrors ours: each task pins a real scientific repository, specifies a typed tool signature in a `task.yaml`, and validates the built tool by *live invocation against held-out gold*. Gold is a mix of strict value equivalence (`np.testing.assert_allclose`, `atol=10-3`, against precomputed reference arrays), exact scalar match (p-values, parameter counts), shape/type contracts, and output-artefact checks; crucially, the *same tool* is

exercised over multiple named input cases with per-case references. The suite spans 15 tasks across 7 domains (pathology, radiology, genomics/proteomics, imaging, 3D vision, tabular, LLMs) with 43 multi-input invocations and 100+ assertions. We note honestly that gold strictness is uneven—four tasks (musk, retfound, nnunet, medsss) carry only type/existence checks—and we report per-task strictness. For breadth beyond TM-Bench’s biomedical lean we draw repositories from ML-Bench (Tang et al. 2024a) and the ToolRosella inventory (Di et al. 2026) (the latter for coverage only, as it ships no gold).

Systems compared. We compare (i) the **frozen catalogue**: tools converted once by ALEMBIC and invoked deterministically, against (ii) a same-model **coding agent** (OpenHands with GLM-5.2) that re-derives each task from scratch. Both use the identical held-out TM-Bench tests with matched denominators; all conversions and solves use the same underlying model.

Metrics. We report four families aligned with the thesis: (1) *coverage / adequacy*—how many repositories convert to gold-passing tools, plus a tool-adequacy audit (are any catalogue tools hollow or hallucinated?); (2) *reproducibility*—determinism of the frozen tool versus the agent’s pass^k and run-to-run *contract drift* (do repeated re-derivations agree in output format and value?); (3) *economy*—per-invocation cost and the break-even number of calls that amortises the one-time conversion across a many-call system; (4) *verification quality*—the fraction of “passing” tools rejected by our mock-guard relative to import-smoke or LLM-judge validation. Operational limits and all runs are released for reproducibility.

6 Results

7 Discussion

8 Limitations

9 Conclusion

References

Anonymous. 2025. MCP at First Glance: Studying the Security and Maintainability of MCP Servers. *arXiv preprint arXiv:2506.13538*.

Bogin, B.; Yang, K.; Gupta, S.; Richardson, K.; Bransom, E.; Clark, P.; Sabharwal, A.; and Khot, T. 2024. SUPER: Evaluating Agents on Setting Up and Executing Tasks from Research Repositories. *arXiv preprint arXiv:2409.07440*.

Boiko, D. A.; MacKnight, R.; Kline, B.; and Gomes, G. 2023. Autonomous chemical research with large language models. *Nature*, 624(7992): 570–578.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2023. Large Language Models as Tool Makers. *arXiv preprint arXiv:2305.17126*.

Chen, Z.; Chen, S.; Ning, Y.; et al. 2025. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.05080.

Di, S.; Yuan, X.; Guo, H.; Ouyang, C.; Liu, Y.; Yue, L.; Chen, Z.; Zheng, L.; Zhu, J.; Pan, S.; Yin, J.; Rui, Y.; and Zhang, M.-L. 2026. ToolRosella: Translating Code Repositories into Standardized Tools for Scientific Agents. *arXiv preprint arXiv:2603.09290*.

Gao, S.; et al. 2025. Democratizing AI Scientists using ToolUniverse. *arXiv preprint arXiv:2509.23426*.

Gottweis, J.; Weng, W.-H.; Daryin, A.; et al. 2025. Towards an AI Co-Scientist. *arXiv preprint arXiv:2502.18864*.

He, H.; and Thinking Machines Lab. 2025. Defeating Non-determinism in LLM Inference. Technical report, Thinking Machines Lab. <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.

Hou, X.; Zhao, Y.; Wang, S.; and Wang, H. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *arXiv preprint arXiv:2503.23278*.

Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*.

M. Bran, A.; Cox, S.; Schilter, O.; Baldassari, C.; White, A. D.; and Schwaller, P. 2024. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6: 525–535. ArXiv:2304.05376.

Miao, J.; and Zou, J. 2025. Paper2Agent: Reimagining Research Papers as Interactive and Reliable AI Agents. *arXiv preprint arXiv:2509.06917*.

Ouyang, C.; Yue, L.; Di, S.; Zheng, L.; Yue, L.; Pan, S.; Yin, J.; and Zhang, M.-L. 2025. Code2MCP: Transforming Code Repositories into MCP Services. *arXiv preprint arXiv:2509.05941*.

Samuel, S.; and Mietchen, D. 2024. Understanding and improving the reproducibility of Jupyter notebooks from biomedical publications. *GigaScience*, 13. ArXiv:2308.07333.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; et al. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*.

Schmidgall, S.; Su, Y.; Wang, Z.; et al. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. *arXiv preprint arXiv:2501.04227*.

Siegel, Z. S.; Kapoor, S.; Nagdir, N.; Stroebel, B.; and Narayanan, A. 2024. CORE-Bench: Fostering the Credibility of Published Research Through a Computational Reproducibility Agent Benchmark. *arXiv preprint arXiv:2409.11363*.

Starace, G.; Jaffe, O.; Sherburn, D.; et al. 2025. Paper-Bench: Evaluating AI’s Ability to Replicate AI Research.

In *International Conference on Machine Learning (ICML)*. ArXiv:2504.01848.

Swanson, K.; Wu, W.; Bulaong, N. L.; Pak, J. E.; and Zou, J. 2025. The Virtual Lab: AI Agents Design New SARS-CoV-2 Nanobodies with Experimental Validation. *Nature*. BioRxiv 2024.11.11.623004.

Tang, X.; Liu, Y.; Cai, Z.; et al. 2024a. ML-Bench: Evaluating Large Language Models and Agents for Machine Learning Tasks on Repository-Level Code. *arXiv preprint arXiv:2311.09835*.

Tang, X.; Qian, B.; Gao, R.; et al. 2024b. BioCoder: A Benchmark for Bioinformatics Code Generation with Large Language Models. *Bioinformatics*, 40. ArXiv:2308.16458.

Tian, M.; Gao, L.; Zhang, S. D.; et al. 2024. SciCode: A Research Coding Benchmark Curated by Scientists. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*. ArXiv:2407.13168.

Wang, G.; Xie, Y.; Jiang, Y.; Mandekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*.

Wölflein, G.; Ferber, D.; Truhn, D.; Arandjelović, O.; and Kather, J. N. 2025. LLM Agents Making Agent Tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. ArXiv:2502.11705.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*.

Yuan, L.; Chen, Y.; Wang, X.; Fung, Y. R.; Peng, H.; and Ji, H. 2024. CRAFT: Customizing LLMs by Creating and Retrieving from Specialized Toolsets. In *International Conference on Learning Representations (ICLR)*. ArXiv:2309.17428.

A Reproducibility and Artefacts

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this .tex file directly.

For each question (that applies), replace the “Type your response here” text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the “Type your response here” text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the \question command or any other lines in this document.

You can \input this .tex file right before \end{document} of your main file or compile it as a stand-alone document. Check the instructions on your conference’s website to see if you will be asked to provide this checklist with your paper or separately.

1. General Paper Structure

- 1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) [Type your response here](#)
- 1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) [Type your response here](#)
- 1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) [Type your response here](#)

2. Theoretical Contributions

- 2.1. Does this paper make theoretical contributions? (yes/no) [Type your response here](#)

If yes, please address the following points:

- 2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) [Type your response here](#)
- 2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) [Type your response here](#)
- 2.4. Proofs of all novel claims are included (yes/partial/no) [Type your response here](#)
- 2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) [Type your response here](#)
- 2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) [Type your response here](#)
- 2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) [Type your response here](#)
- 2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) [Type your response here](#)

3. Dataset Usage

3.1. Does this paper rely on one or more datasets? (yes/no) [Type your response here](#)

If yes, please address the following points:

3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) [Type your response here](#)

3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) [Type your response here](#)

3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) [Type your response here](#)

3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations (yes/no/NA) [Type your response here](#)

3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) [Type your response here](#)

3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisfying (yes/partial/no/NA) [Type your response here](#)

4. Computational Experiments

4.1. Does this paper include computational experiments? (yes/no) [Type your response here](#)

If yes, please address the following points:

4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) [Type your response here](#)

4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) [Type your response here](#)

4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) [Type your response here](#)

4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) [Type your response here](#)

4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from

(yes/partial/no) [Type your response here](#)

4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) [Type your response here](#)

4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) [Type your response here](#)

4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) [Type your response here](#)

4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) [Type your response here](#)

4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) [Type your response here](#)

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) [Type your response here](#)

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) [Type your response here](#)